AISA-ArabicFC: The First Shared Task on Arabic Function Calling for Agentic AI Systems

Omer Nacar[1*], Wareef Alzubaidi[1], Mohammed Alkhalifah[1]

[1] Tuwaiq Academy

o.najar@tuwaiq.edu.sa wareefalharbi51@gmail.com m.alkhalifa@tuwaiq.edu.sa

Abstract

Function calling connects language models to external tools, but Arabic structured tool use remains insufficiently evaluated. We present AISA-ArabicFC, a shared task at ArabicNLP 2026 built around 12,220 Arabic requests spanning five dialect groups and eight service domains. Each request has four candidate schemas drawn from a registry of 27 tools, including seven tools that never occur as gold calls; 223 requests require abstention. Systems decide whether to call, select a function, and extract its arguments. Two ranked tracks evaluate core calling and reasoning-trace compliance, with a third diagnostic examining dialect robustness. Twenty-two teams submitted to the blind phase, and 21 produced valid predictions on 1,125 test requests. We analyze 73 submissions and 13 system descriptions. A LoRA-adapted 4B model ranks first on both competition tracks, with scores of 0.9279 and 0.9400, while argument exact match peaks at 0.8816. However, a deterministic rule combining candidate order and released metadata matches the best observed function-name accuracy of 0.9973, limiting what this metric establishes about language understanding. Exact argument extraction provides most of the competitive separation. Maghrebi has the lowest score for every system, although its small test subset and differences in example composition limit interpretation. We provide public task data, a baseline checkpoint, the scorer, and final rankings, and document evaluation shortcuts and remaining reproducibility limitations.[1]

1 Introduction

Large language models increasingly serve as the *action layer* of agentic systems, converting natural-language requests into structured calls that external runtimes can validate and execute. Yet reliable function calling remains difficult under realistic schemas (Patil et al., 2025; Guo et al., 2024), and performance degrades further outside English; MASSIVE-Agents (Kulkarni et al., 2025) reports substantial cross-lingual disparities across 52 languages.

Arabic NLP has strong pretrained and instruction-tuned models (Antoun et al., 2020; Abdul-Mageed et al., 2021; Sengupta et al., 2023; Huang et al., 2024a). Evaluating structured tool use nevertheless requires more than measuring Arabic text understanding or generation. A system must identify whether a request warrants a call, choose an appropriate function, and populate the expected arguments. AISA-ArabicFC provides a shared evaluation of these components using a blind submission phase, a public scorer, and comparisons across participant systems.

Task design. An instance is one Arabic user request together with four candidate tool schemas. A schema specifies a function's name, purpose, argument fields, and value constraints. For example, a currency-conversion function needs an amount and source and target currencies; identifying the function alone is insufficient. The task has three prediction subtasks: *decide* whether a call is needed, *select* its function, and *extract* the annotated arguments. Each instance has at most one gold call; otherwise, the correct output is abstention. These subtasks are shared across the evaluation tracks. Track A evaluates function-name accuracy (FnAcc) and normalized argument exact match (ArgEM). Track B evaluates the same outputs and additionally rewards the presence of a reasoning trace (ThinkRate). Track C is a diagnostic that averages Track A scores across the five dialect groups. Section 3 defines the metrics and

[1] https://huggingface.co/spaces/TuwaiqAcademy/AISA-ArabicFC-Shared-Task

Proceedings of The Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026) - Volume 2 Shared Tasks, pages 1531–1546
October 28-29, 2026 ©2026 Association for Computational Linguistics

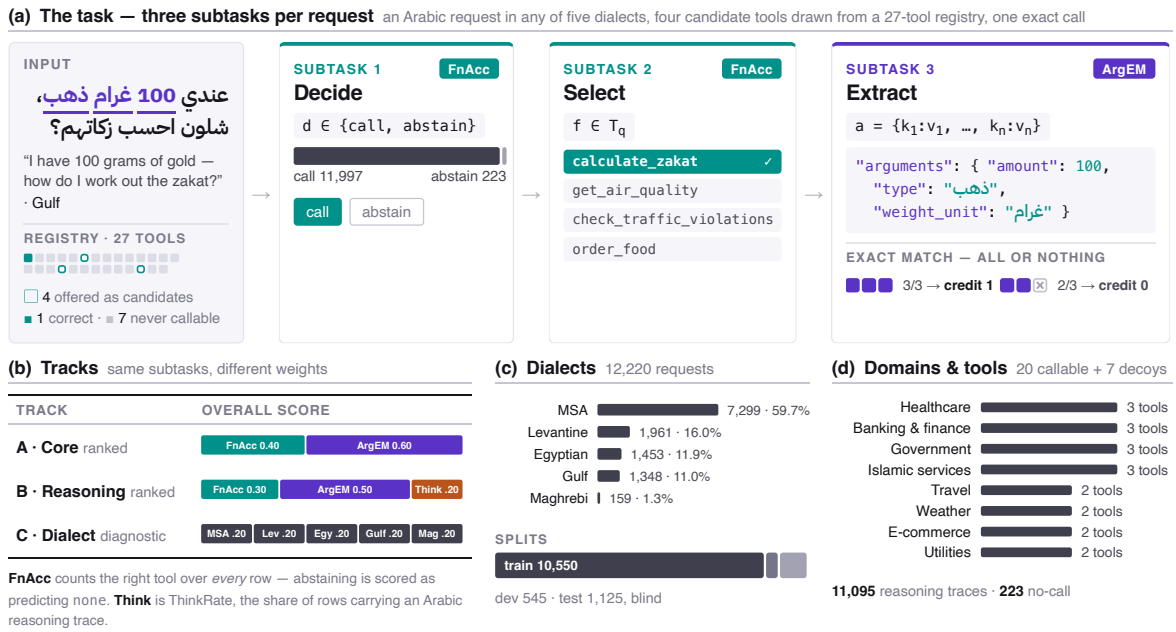


Figure 1: Overview of the AISA-ArabicFC shared task. Each request requires either one structured function call or abstention. Tracks A and B evaluate submitted predictions; Track C provides a dialect-level diagnostic.

their weights. Figure 1 summarizes the setup.

Our contributions.

- **An Arabic function-calling shared evaluation.** We organize a shared task over 12,220 requests, five dialect groups, eight domains, and 27 tool schemas, including 223 no-call instances and 11,095 released training and development reasoning traces.
- **Complementary evaluation views.** Two ranked tracks assess function selection, argument extraction, and reasoning-trace compliance; a dialect diagnostic exposes variation hidden by aggregate scores.
- **Results and benchmark diagnostics.** We analyze 21 valid team entries and 13 system descriptions, identify argument extraction as the main source of competitive separation, and quantify candidate-order and metadata shortcuts that qualify the interpretation of function-name accuracy.

2 Related Work

Tool-calling benchmarks. ToolLLM and ToolBench (Qin et al., 2024), ToolQA (Zhuang et al., 2023), StableToolBench (Guo et al., 2024), UltraTool (Huang et al., 2024b), and BFCL (Patil et al., 2025) establish structured tool use as a core evaluation problem, exposing schema, hallucination, argument, and abstention failures. These benchmarks are largely English-first; MASSIVE-Agents (Kulkarni et al., 2025) extends evaluation to 52 languages but does not target Arabic tool use, dialectal variation, or production-style schemas. AISA-ArabicFC addresses these dimensions directly.

Arabic NLP and function calling. Arabic NLP has produced strong encoders and generative models, including AraBERT (Antoun et al., 2020), ARBERT/MARBERT (Abdul-Mageed et al., 2021), Jais (Sengupta et al., 2023), AceGPT (Huang et al., 2024a), and ALLaM (Bari et al., 2025), alongside advances in Arabic representation learning and embeddings (Elshehy et al., 2026; Nacar and Koubaa, 2025; Nacar et al., 2025). These systems improve understanding and generation but do not directly evaluate whether a model can emit an executable, schema-conformant action. That distinction motivates AISA-ArabicFC: fluent Arabic understanding does not by itself imply reliable tool selection, abstention, or argument realization.

Interfaces, compact backbones, and agentic evaluation. Closed APIs expose native function-calling interfaces (OpenAI, 2024; Anthropic, 2026), while compact open models can be specialized through parameter-efficient adaptation such as LoRA (Hu et al., 2021). The released model card identifies our 270M-parameter AISA-ARFunctionCall-Think baseline as a LoRA adaptation of Gemma 3 (Team et al., 2025). Strong

Dialect	Train	Dev	Test (released)	Test (Track C gold)	Total	%
MSA	6,151	323	825	660	7,299	59.7
Levantine	1,784	75	102	128	1,961	16.0
Egyptian	1,283	66	104	175	1,453	11.9
Gulf	1,190	75	83	128	1,348	11.0
Maghrebi	142	6	11	34	159	1.3
Total	**10,550**	**545**	**1,125**	**1,125**	**12,220**	**100**
No-call rows	50	45	128	128	223	1.8

Table 1: Corpus composition. Dialect totals and percentages use the released labels. The separate final-gold test column shows the labels used for Track C after 179 dialect reassignments; test membership is unchanged. No-call counts use the final test function labels. Dialect relabeling itself does not change call labels, but separate gold revisions did change function labels.

adapted open-model submissions motivate further study of task-specific supervision; however, differences between submitted pipelines prevent isolating the effects of model size or backbone. More broadly, agentic systems require orchestration, evaluation, and governance (Wu et al., 2023; Anthropic, 2024; Model Context Protocol, 2025; Liu et al., 2024; AI, 2023; International Organization for Standardization, 2023b,a); AISA-ArabicFC addresses structured Arabic prediction within that broader setting.

3 Task Definition

Inputs and outputs. A tool schema describes a function's name, intended operation, argument names and types, and any required fields or allowed values. Let $\mathcal{T}$ contain the 27 registry schemas. Each instance (q, T_q) contains an Arabic request q and exactly four candidate schemas $T_q \subset \mathcal{T}$. Positive instances have one gold function among the candidates; negative instances require no call. The setting is single-turn and single-call: it does not evaluate requests requiring several tools or retrieval from an unrestricted tool registry.

Systems perform three related subtasks:

- **Decide:** determine whether the request requires a call.
- **Select:** predict the function name if a call is needed.
- **Extract:** predict the argument dictionary specified by the annotation policy.

The policy includes information explicitly stated or unambiguously derivable from the request; absent information should not be invented solely to fill a schema field. Evaluation compares predictions with the reference dictionary under the official normalizer. It does not execute the call or establish that a downstream service would accept it.

The four candidates define the available choice set, rather than the number of functions semantically related to a request. Twenty registry tools occur as gold calls; seven occur only as distractors. This restricted setting reduces selection difficulty. Candidate order and additional released metadata create further shortcuts, quantified in Section 7.1.

Submissions use JSON Lines with one object per test instance containing `id`, `tool_called`, and `arguments`; Track B additionally uses `think`. Predicting the reserved function label `none` denotes abstention, so the call decision is represented through the function-name output rather than a separate scored field.

Metrics. Let N be the number of evaluated rows and let $\mathcal{P}$ contain the N_+ rows requiring a call. Let f_i, a_i, and t_i denote a function label, argument dictionary, and trace, respectively; hats indicate predictions. Define $\equiv_{\mathcal{N}}$ as equality under the official argument normalizer. The scorer computes

$$\textsc{FnAcc} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{f}_i = f_i], \tag{1}$$

$$\textsc{ArgEM} = \frac{1}{N_+}\sum_{i\in\mathcal{P}} \mathbb{1}[\hat{a}_i \equiv_{\mathcal{N}} a_i], \tag{2}$$

$$\textsc{ThinkRate} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[|\operatorname{strip}(\hat{t}_i)| > 5]. \tag{3}$$

FnAcc includes negative rows, whose reference label is `none`. It therefore penalizes false calls and missed calls as well as incorrect tool names. ArgEM is an all-or-nothing dictionary match on positive rows and is computed independently of function-name correctness; it is not a joint measure of a fully correct call. Missing predictions receive no credit. For ThinkRate, only a string containing more than five characters after stripping surrounding whitespace qualifies. Although

Track B requests Arabic reasoning, the scorer does not verify its language, factual quality, generation order, or contribution to the prediction.

Argument normalization. The published normalizer maps selected representational variants to common forms before comparison. It handles Arabic-Indic and Western digits, diacritics and *tatweel*, selected Arabic orthographic variants, numeric representations, documented list-field comparisons, and field-specific aliases. These conventions reduce penalties for some equivalent forms but do not establish general semantic equivalence or runtime schema validity. The same final scorer and normalizer are used for all reported final scores.

Tracks and weights. Track A ranks core calling with

$$S_A = 0.40\,\textsc{FnAcc} + 0.60\,\textsc{ArgEM}. \quad (4)$$

Track B adds trace compliance:

$$S_B = 0.30\,\textsc{FnAcc}{+}0.50\,\textsc{ArgEM}{+}0.20\,\textsc{ThinkRate}. \quad (5)$$

These weights were specified in the task proposal and published before the blind evaluation. They express the organizers' decision to give argument extraction greater weight as the primary challenge; they were not optimized against the final submissions, and no claim of weight optimality is made.

Track C is a dialect diagnostic:

$$S_C = \frac{1}{5}\sum_{g\in\mathcal{D}} \left(0.40\,\textsc{FnAcc}_g + 0.60\,\textsc{ArgEM}_g\right), \quad (6)$$

where $\mathcal{D}$ contains MSA, Gulf, Egyptian, Levantine, and Maghrebi. Function accuracy within a dialect uses all its rows; argument exact match uses only its positive rows. This equal-dialect average differs from weighting by dialect frequency. The ordered Track C table is a descriptive comparison; Tracks A and B are the ranked competition tracks.

4 Data

Provenance and construction. The corpus derives from the Arabic Function Calling collection (Haroon, 2024). Preparation included schema and argument corrections, value canonicalization, four-candidate tool sets, and no-call examples. Versioned scripts and a changelog document successive annotation-policy changes. However, the surviving records do not fully establish how dialectal requests, abstention examples, and reasoning traces were originally produced. We therefore do not infer a human-only or LLM-assisted generation procedure from the evaluation artifacts.

Composition. Table 1 reports 10,550 training, 545 development, and 1,125 test requests. Using the final test call labels, the corpus contains 11,997 positive and 223 no-call instances. No-call counts are 50, 45, and 128 across the three splits, corresponding to 0.47%, 8.26%, and 11.38%; abstention prevalence therefore differs substantially across splits. The no-call examples span eleven recorded categories, including non-actionable domain questions, completed actions, hypothetical requests, and meta-questions. The training and development splits provide 11,095 Arabic reasoning traces in total. Test reference traces and gold calls were withheld during the blind phase.

Tool registry. The registry contains 27 schemas across eight service domains, of which 20 occur as gold calls and seven occur only as distractors (Table 3). Each request is presented with four candidate schemas. This defines a restricted candidate-selection setting; the candidate-order and metadata shortcuts documented in Section 7.1 limit conclusions about semantic discrimination between tools.

5 Baselines and Evaluation

Platform and evaluation phases. The competition used a custom Hugging Face Space for development submissions and final results.[2] During development, the leaderboard scored predictions against the development split and returned results immediately. During the blind phase, participants received test inputs without reference calls, and the portal collected submissions and validated their format without returning test scores. The organizers scored submissions offline against the final gold and published the results after the phase closed.

Submissions and selection. Twenty-two teams submitted to the blind phase. One team's files for both ranked tracks contained no parseable predictions and were recorded as invalid with zero scores; 21 teams therefore appear in the valid-result tables. The submission record contains 73 entries. Teams could submit at most three runs per ranked track. For each team, the published Track A and Track B

[2] https://huggingface.co/spaces/TuwaiqAcademy/AISA-ArabicFC-SharedTask-Leaderboard

System	FnAcc	ArgEM	A	B
GPT-4o, zero-shot	0.9324	0.7442	0.8195	n/a
GPT-4o, 3-shot	0.8676	0.6991	0.7665	n/a
AISA-AR-FunctionCall-Think 270M	0.9813	0.6790	0.8000	0.8074
Archived random run	0.0418	0.0060	0.0203	0.0155
Participant median	**0.9893**	**0.8506**	**0.8928**	**0.9106**
Participant best	**0.9973**	**0.8816**	**0.9279**	**0.9400**

Table 2: Archived baseline predictions rescored against the final test gold with the official scorer. Participant reference points are column-wise summaries, using Track A entries for its component metrics and each track's entries for its Overall score. GPT-4o was not prompted for traces. The random run is retained as a historical result; its generation procedure was not recovered and it is not an established four-candidate chance baseline.

Domain	Tools	Examples
Healthcare	3	appointments, medication
Banking & finance	3	transfer, currency
Government	3	iqama, visa, violations
Islamic services	3	zakat, qibla, Quran
Travel	2	hotels, umrah packages
Weather	2	forecast, air quality
E-commerce	2	price comparison, food
Utilities	2	translation, end of service
Callable	**20**	
Distractors	7	never the correct answer

Table 3: The 27-tool registry. Four schemas are shown as candidates per request.

entries are the runs with the highest respective track score. The two track tables each contain 17 valid teams, with 13 teams in common and four teams unique to each track.

Baselines. Table 2 reports scores recomputed from four archived prediction files using the same final gold and scorer as the participant evaluation. The released AISA-AR-FunctionCall-Think checkpoint contains 270M parameters. Its model card identifies Gemma 3 as the base model and describes LoRA adaptation with rank 64, three training epochs, and a cosine learning-rate schedule.[3] The original complete training configuration was not recovered; these model-card details should not be read as a complete retraining specification.

For GPT-4o, the retained inference script uses the unpinned API alias `gpt-4o`, temperature 0, and a maximum of 300 output tokens. The three-shot condition uses three manually authored demonstrations, including one no-call example, rather than examples sampled from the training split. The script specifies `max_retries=3` with exponential backoff, removes Markdown code fences, and uses a regular-expression fallback to recover JSON. Its documented failure default is `none`. The original API snapshot cannot be recovered from the alias alone. Appendix E records the recoverable configuration and its limits.

The archived random predictions also reproduce their published scores, but the generator is unavailable. We retain this row for historical transparency and do not describe it as uniform candidate sampling. With 997 positive rows, uniform sampling among four candidates and no abstention would instead have expected FnAcc of $(997/1125)/4 = 0.2216$.

Baseline results. The 270M baseline has higher FnAcc than zero-shot GPT-4o (0.9813 versus 0.9324), whereas GPT-4o has higher ArgEM (0.7442 versus 0.6790). GPT-4o therefore obtains the stronger baseline Track A score, 0.8195. Its three-shot configuration scores 0.7665, showing that this particular set of authored demonstrations did not improve the reported run. This single comparison does not establish a general disadvantage of few-shot prompting. The best participant reaches ArgEM 0.8816. Candidate-order and metadata shortcuts limit the interpretation of high FnAcc independently of argument extraction (Section 7.1).

6 Participant Systems

We summarize 13 system descriptions associated with teams in the valid-result tables. The leaderboards contain 21 valid team entries in total; for the remaining eight, available metadata may identify a model or approach, but does not support the same level of methodological analysis.

The submissions differ on one axis far more than on any other. Mean function name accuracy in Track A was near the ceiling at 0.9749, though for reasons partly attributable to the benchmark rather than to the systems, while argument exact match averaged 0.8185 over a range nearly three times

[3] https://huggingface.co/TuwaiqAcademy/AISA-AR-FunctionCall-Think

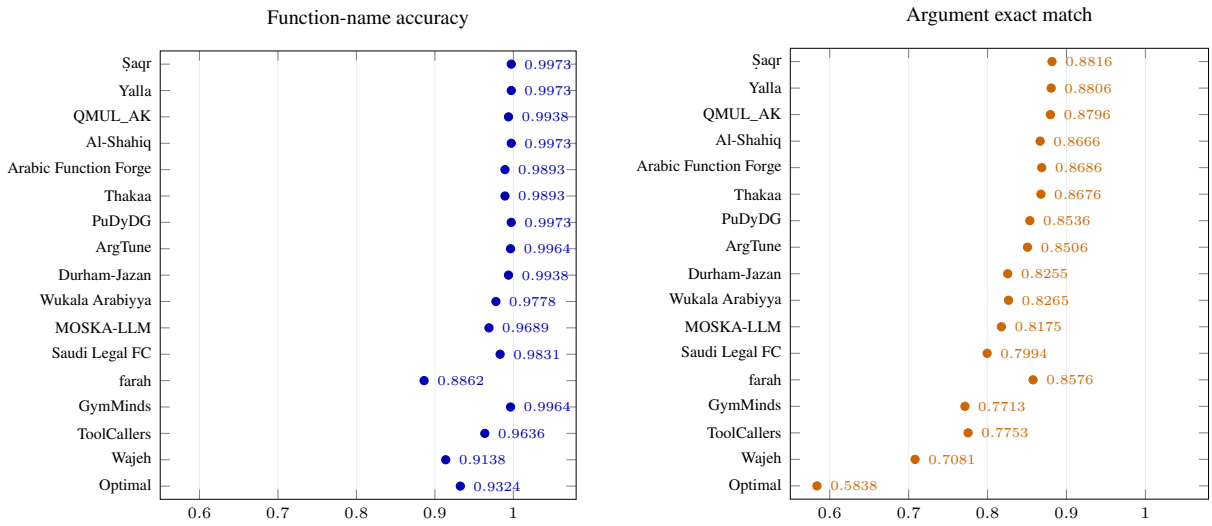


Figure 2: Component scores for all 17 Track A systems in official rank order. The panels share the same horizontal range. High function-name accuracy must be interpreted in light of the input shortcuts in Section 7.1.

as wide. Figure 2 shows how far the two metrics diverge: teams were separated almost entirely by how they wrote arguments rather than by which tool they chose.

6.1 Fine-Tuned Open Models

Ten of the thirteen system papers adapted open models, and this family occupied the top three places. Ṣaqr (Khaled Abdelfattah et al., 2026) won with a single LoRA-tuned Qwen3-4B trained for two epochs on the provided data; inference-time reasoning prefill and schema-typed parsing added 0.0063 to Overall A and 0.0223 to Overall B without retraining. Yalla (Joshi, 2026) placed second with three diverse Qwen 4B/7B LoRA models combined by per-argument voting; supervising only the scored object, 30.3% of the distributed target, yielded its largest measured gain of +0.020 ArgEM. QMUL_AK (Khairallah and Zubiaga, 2026) placed third on Tracks A/B and first on Track C using LoRA-tuned ALLaM-7B with schema-aware prompting, token-weighted loss, deterministic validation, and no-call/minority-dialect resampling, achieving the smallest dialect gap (0.077).

Two teams built directly on the released 270M baseline. PuDyDG (Dang Phu and Dang Van, 2026) decomposed training into five objectives and added 500 validated function-centric synthetic examples, while ArgTune (Baothman, 2026) separated tool selection from general and specialist argument extraction, reaching eighth place on T4 GPUs. Durham-Jazan (Alshehri et al., 2026) combined six QLoRA adapters with field-wise voting and train-derived rules on a single 16 GB GPU; Saudi Legal FC (Almohammedi, 2026) used completion-only loss masking, improving development ArgEM from 0.732 to 0.778; and Siraj (Alanazi and Alshammari, 2026) added a separately trained argument repairer behind a conservative acceptance filter. Makeen (Alzahrany et al., 2026) combined a label-semantic span extractor with a hint-grounded generator and confidence-gated argument routing and additionally released 5,758 Arabic tool-use examples covering 1,442 tools for unseen-tool generalization. Optimal (Alowjan et al., 2026) paired a 4-bit QLoRA Qwen2.5-7B with deterministic parsing, validation, and type casting, shifting schema compliance from the model to a post-processing layer.

6.2 Training-Free Systems

Among the teams whose system descriptions we analyze, Arabic Function Forge and Thakaa submitted prompt-only systems and placed fifth and sixth in Track A, respectively. Arabic Function Forge (Alshehri and Aljubayri, 2026) prompted a single off-the-shelf open-weights model once per request. Its error analysis identified over-interpretation as a dominant failure mode, including translating argument values, converting dates to ISO form, and supplying fields absent from the request. Its prompt explicitly discouraged these transformations.

Thakaa (Alqaeri et al., 2026) placed sixth with a prompt-only pipeline over gemma-4-31B-it and separately analyzed a gemma-4-12B-it fine-tune that was never scored on the test. Two results follow. Stating the annotation conventions outright is worth

Rank	Team	Overall	FnAcc	ArgEM	Paper	Approach	Base model(s)
1	Ṣaqr	0.9279	0.9973	0.8816	Yes	Fine-tuned	Qwen3-4B-Instruct-2507
2	Yalla	0.9273	0.9973	0.8806	Yes	Ensemble	Qwen3-4B + Qwen2.5-7B
3	QMUL_AK	0.9253	0.9938	0.8796	Yes	Fine-tuned	ALLaM-7B-Instruct-preview
4	Al-Shahiq	0.9189	0.9973	0.8666	–	Prompting	GPT-5.6 (API)
5	Arabic Function Forge	0.9169	0.9893	0.8686	Yes	Prompting	Qwen3.5-122B-A10B
6	Thakaa	0.9163	0.9893	0.8676	Yes	Prompting	gemma-4-31B-it
7	PuDyDG	0.9111	0.9973	0.8536	Yes	Fine-tuned	AISA-AR-FC-Think
8	ArgTune	0.9089	0.9964	0.8506	Yes	Fine-tuned	AISA-AR-FC-Think
9	Durham-Jazan	0.8928	0.9938	0.8255	Yes	Ensemble	ALLaM-7B Qwen2.5-7B
10	Wukala Arabiyya	0.8870	0.9778	0.8265	–	Fine-tuned	Aya 8B
11	MOSKA-LLM	0.8780	0.9689	0.8175	–	Ensemble	MARABERTv2 + gemma-4-12B
12	Saudi Legal FC	0.8729	0.9831	0.7994	Yes	Fine-tuned	Qwen2.5-7B-Instruct
13	farah	0.8690	0.8862	0.8576	–	–	–
14	GymMinds	0.8614	0.9964	0.7713	–	–	–
15	ToolCallers	0.8506	0.9636	0.7753	–	–	–
16	Wajeh	0.7904	0.9138	0.7081	Yes	Other	TF-IDF + log. reg.
17	Optimal	0.7232	0.9324	0.5838	Yes	Fine-tuned	Qwen2.5-7B-Instruct
	GPT-4o, zero-shot	*0.8195*	*0.9324*	*0.7442*		*Prompting*	*GPT-4o*
	GPT-4o, 3-shot	*0.7665*	*0.8676*	*0.6991*		*Prompting*	*GPT-4o*
	AISA-Think	*0.8000*	*0.9813*	*0.6790*		*Fine-tuned*	*Gemma 3 270M + LoRA*
	Archived random run	*0.0203*	*0.0418*	*0.0060*		*Unrecovered*	–

Table 4: Track A leaderboard, ranked by $0.40\,\mathrm{FnAcc} + 0.60\,\mathrm{ArgEM}$. The grouped, italicized baseline rows use the same order as Table 2 and are not competition entries. All displayed scores use the final gold and scorer. The archived random generator is unavailable. Dashes indicate unavailable or non-applicable information.

7.5 ArgEM points to a model that has not seen them and nothing to one that has. Its search over rules, retrieval, prompting, selection, reranking, and fusion did not yield a gain that survived the team's replication procedure. This finding concerns the tested configurations and development data; it does not establish a universal minimum detectable effect from the split size alone.

6.3 A System Without a Language Model

Wajeh (Shehri and Aldosari, 2026) submitted NABIQ, which contains no large language model, no neural network, no external API, and no GPU. It routes tools by character n-gram TF-IDF with logistic regression, populates arguments by per-tool nearest-neighbor retrieval over training data, and gates every value through an evidence test: a candidate may be emitted only if it is a normalized substring of the utterance, a member of a closed canonical map, or a date whose tokens occur in the text. It reached 0.7904 overall on CPU, and its methodological result is discussed in Section 7.

6.4 Reasoning Traces and Dialect Handling

Across Track B, the trace requirement was met almost universally, with 12 of the 17 scored systems emitting a trace on every instance. How it was produced varied more than the scores suggest. Saudi Legal FC generates it in a single autoregressive output sequence, emitting an Arabic `think` field inside the same JSON object as the call, so that abstention on a negative instance carries an explicit rationale. Others attached it separately, and two teams satisfied the requirement by construction, a point taken up in Section 8 and in the Limitations.

Several teams considered or incorporated dialect-related strategies, but only QMUL_AK explicitly reports dialect-specific handling as part of the submitted system and links it to the intended robustness objective. QMUL_AK resampled minority-dialect examples during training and is the only top-three Track A system to have done so. One team investigated its own dialect gap rather than acting on it: Saudi Legal FC found that Levantine, its weakest dialect, was already over-represented in training relative to development, so under-exposure could not explain the deficit, and a manual review of sixteen Levantine failures attributed twelve to the enumeration and date-format patterns that also dominate its MSA failures. It therefore made no dialect-specific intervention, concluding instead that the observed gap reflected recurring formatting patterns rather than insufficient dialect exposure, consistent with Section 8.

7 Results

We report 17 systems each for Tracks A and B and 21 for Track C, which covers all valid teams entering

either ranked track. The Track A leaderboard appears in the main text; complete Track B and Track C results are provided in Appendices A and B, respectively.

7.1 Track A: Core Function Calling

Table 4 shows a closely grouped leading set: Ṣaqr scores 0.9279, Yalla 0.9273, and QMUL_AK 0.9253. Their argument dictionaries match on 879, 878, and 877 of the 997 positive rows, respectively. Ṣaqr and Yalla have the same number of correct function labels, so their score difference is exactly one net argument match, contributing $0.60/997 = 0.000602$ to Track A. This is a difference in total correct counts; it does not establish that the systems disagree on only one row or are statistically equivalent.

Across all 17 Track A systems, ArgEM ranges from 0.5838 to 0.8816, and 15 systems exceed zero-shot GPT-4o's 0.7442. Argument extraction therefore provides substantial separation under the final normalization policy. However, the near-ceiling function scores require a different interpretation.

Candidate-order and metadata shortcuts. The gold tool occupies the first candidate position on all 997 positive test rows. Always choosing that position therefore gives FnAcc $997/1125 = 0.8862$. In addition, the released `negative_category` field is non-null on 125 rows, all negative under the final gold; it identifies $125/128 = 97.66\%$ of final no-call rows without any false positives. A rule that predicts `none` when this field is non-null and otherwise chooses the first candidate consequently obtains

$$\text{FnAcc}_{\text{shortcut}} = \frac{125 + 997}{1125} = 0.9973. \quad (7)$$

This analytical count matches the best observed participant FnAcc without interpreting the request. Table 5 reports these label-only diagnostics; neither rule supplies an argument dictionary. The result does not establish that any participant used the shortcut, but it prevents interpreting the observed maximum as evidence that Arabic tool selection is solved.

Abstention diagnostic. For QMUL_AK, inspection of the archived submission attributes all seven function-label errors to the call decision: six false calls on 128 negative rows and one missed call on 997 positive rows. The corresponding rates are 4.69% and 0.10%. There are no wrong-tool errors among positive rows for which the system predicts a call. This localizes this system's function-label errors, but the released shortcuts prevent attributing the remaining correctness solely to semantic understanding, and this single-system analysis does not characterize all entrants.

Label-only rule	Correct	FnAcc
First candidate	997/1125	0.8862
Metadata + first candidate	1122/1125	0.9973

Table 5: Analytical function-label diagnostics derived from the released input properties and final labels. These are not argument-prediction systems or ranked submissions.

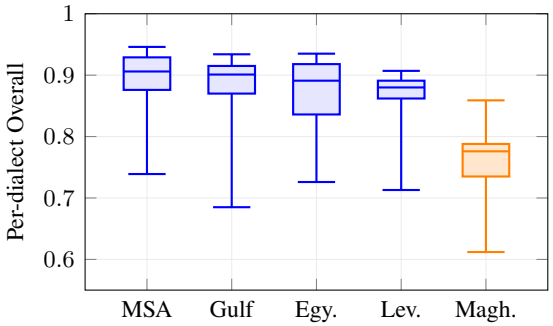


Figure 3: Per-dialect Overall scores across the 21 Track C systems. Boxes show the interquartile range, internal lines show medians, and whiskers show minima and maxima. Maghrebi has the lowest score for every system; its maximum (0.859) is below the median of each other dialect group.

7.2 Track B: Reasoning Function Calling

Track B preserves the top ordering from Track A: Ṣaqr ranks first at 0.9400, followed by Yalla at 0.9395 and QMUL_AK at 0.9380. Reasoning-trace compliance is nearly saturated, with 12 of 17 systems reaching ThinkRate = 1.0, so most ranking separation again comes from ArgEM rather than trace presence. Four teams entered Track B without a Track A submission. The full leaderboard is reported in Appendix A.

8 Discussion

Similar scores, different pipelines. Ṣaqr uses one adapted 4B model with reasoning prefill and schema-typed parsing, whereas Yalla combines three models through per-argument voting. Their Track A difference is one net argument match; paired predictions are needed to assess error overlap and uncertainty. Likewise, the close scores of Ṣaqr and QMUL_AK do not isolate backbone ef-

fects: besides using different models, their pipelines differ in sampling, loss weighting, prompting, and validation.

Approach families. Adapted models occupy the top three Track A positions, followed by three prompt-only systems. Wajeh achieves 0.7904 without a neural language model. These results demonstrate several competitive approaches within the fixed-registry setting, but unstandardized training and inference budgets prevent efficiency comparisons. Candidate-order and metadata shortcuts also limit conclusions about semantic tool selection and generalization beyond this setting.

Argument supervision and output conventions. Yalla reports a 0.020 absolute ArgEM gain from supervising only the scored object; Saudi Legal FC improves development ArgEM from 0.732 to 0.778 with completion-only loss; and Thakaa gains 7.5 percentage points by stating annotation conventions explicitly. These within-team effects are not directly comparable, but support attention to supervision and output representation. Participant error analyses similarly identify incorrect values despite correct tools or keys, alongside translation, date rewriting, and unsupported field completion. However, reference mismatches need not imply invalid executable actions; multiple references or execution checks could distinguish acceptable alternatives from errors.

Dialect and trace diagnostics. QMUL_AK obtains the highest dialect macro score and uses minority-dialect resampling, whose contribution was not isolated by ablation. All systems score lowest on Maghrebi, motivating larger dialect subsets matched for tool and argument composition. Meanwhile, 12 of 17 Track B systems attain full trace compliance, sometimes through separately attached traces. Because the scorer checks length rather than language, consistency, or faithfulness, these results do not establish that reasoning improves calls. Appendix C provides detailed supporting evidence.

9 Conclusion

AISA-ArabicFC provides a shared evaluation of Arabic function selection, abstention, and argument extraction across five dialect groups. Twenty-one teams produced valid blind-test predictions, and compact adapted models and prompt-only systems both achieved competitive results. Exact argument extraction supplies most of the separation among leading entries, while candidate-order and metadata shortcuts substantially limit the interpretation of near-ceiling function-name accuracy. The dialect diagnostic consistently identifies lower Maghrebi scores, subject to small-sample and composition effects, and the reasoning track primarily measures trace presence. These results and design findings motivate cleaner input releases, better documented annotation, broader dialect coverage, and evaluation that separates argument grounding, valid execution, and reasoning quality.

Limitations

AISA-ArabicFC evaluates single-turn requests with at most one call and four candidates from a fixed registry, limiting conclusions about multi-tool planning, unrestricted retrieval, and external execution. Candidate ordering and label-proxy metadata jointly enable a deterministic rule to match the best reported FnAcc without interpreting requests, although this does not establish that participants used these shortcuts. Annotation and normalizer revisions limit comparisons across versions; all final scores use the same final gold and scorer. Incomplete records of data generation, reviewer identities and expertise, and independent annotations prevent a complete construction account or a verified annotator count and agreement statistic. Track C uses corrected dialect labels that differ from the released inputs. Its small Maghrebi subset (34 rows), uncontrolled differences in example composition, and the absence of paired uncertainty estimates limit interpretation of dialect differences and close rankings. Furthermore, ThinkRate checks trace length rather than language, faithfulness, or contribution to prediction, while ArgEM evaluates normalized arguments independently of function correctness and does not establish executable-call validity. Reproducibility is constrained by the unpinned GPT-4o alias, unavailable random-baseline generator, and incomplete original 270M training configuration. Independent rescoring requires access to the exact final gold, predictions, scorer, and normalizer revisions.

Acknowledgements

We thank all registered teams, the 22 teams that submitted to the blind phase, and the reviewers for their feedback. Tuwaiq Academy organized and sponsored the task.

References

Muhammad Abdul-Mageed, AbdelRahim Elmadany, et al. 2021. Arbert & marbert: Deep bidirectional transformers for arabic. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)*, pages 7088–7105.

NIST AI. 2023. Artificial intelligence risk management framework (ai rmf 1.0). *URL: https://nvlpubs. nist. gov/nistpubs/ai/nist. ai*, 1(0):0–1.

Abdullah G. Alanazi and Reem K. Alshammari. 2026. Siraj at AISA-ArabicFC: Selective argument repair and the limits of post-editing in Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Abdullah Almohammedi. 2026. Saudi legal FC at AISA-ArabicFC: Disciplined fine-tuning and gold-free post-processing for Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Sarah Alowjan, Lojain Al Mana, and Haya Buayyan Alwizrah. 2026. Optimal at AISA-ArabicFC: A hybrid approach to Arabic function calling with PEFT large language models and deterministic post-processing. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Hassan Alqaeri, Meshal Alamr, and Abdullah Aldahlawi. 2026. Thakaa at AISA-ArabicFC: Convention-aligned prompting and the limits of argument exact match in Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Amal Saad Alshehri, Nelly Bencomo, Housam Babiker, and Amir Atapour-Abarghouei. 2026. Durham-jazan at AISA-ArabicFC: A field-wise QLoRA ensemble and the sensitivity of exact-match scoring. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Anfal Alshehri and Jumanah Aljubayri. 2026. Arabic function forge at AISA-ArabicFC: Copy, don't interpret — training-free Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Nouf Alzahrany, Abir Korched, and Bassam Mattar. 2026. Makeen at AISA-ArabicFC: Grounding LLM via extracted spans and generalization trade-off in Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Anthropic. 2024. Introducing the model context protocol. https://www.anthropic.com/news/model-context-protocol.

Anthropic. 2026. Tool use with claude. https://platform.claude.com/docs/en/agents-and-tools/tool-use/overview. Claude Platform Documentation, accessed September 4, 2026.

Wissam Antoun, Fady Baly, and Hazem Hajj. 2020. Arabert: Transformer-based model for arabic language understanding. In *Proceedings of the 4th workshop on open-source arabic corpora and processing tools, with a shared task on offensive language detection*, pages 9–15.

Sabah Abdulgader Baothman. 2026. ArgTune at AISA-ArabicFC: A modular pipeline for Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

M Saiful Bari, Yazeed Alnumay, Norah Alzahrani, Nouf Alotaibi, Hisham Alyahya, AlRashed AlRashed, Faisal Mirza, Shaykhah Alsubaie, Hassan Alahmed, Ghadah Alabduljabbar, et al. 2025. Allam: Large language models for arabic and english. In *International Conference on Learning Representations*, volume 2025, pages 34179–34214.

Duy Dang Phu and Thin Dang Van. 2026. PuDyDG at AISA-ArabicFC: Decomposed fine-tuning with function-centric few-shot augmentation for Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Omar Elshehy, Omer Nacar, Abdelbasset Djamai, Muhammed Ragab, Khloud AL Jallad, and Mona Abdelazim. 2026. AraModernBERT: Transtokenized initialization and long-context encoder modeling for Arabic. In *Proceedings of the 2nd Workshop on NLP for Languages Using Arabic Script*, pages 313–321.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. 2024. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 11143–11156.

Hesham Haroon. 2024. Arabic function calling dataset. Hugging Face Datasets, https://huggingface.co/datasets/HeshamHaroon/Arabic_Function_Calling.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

Huang Huang, Fei Yu, Jianqing Zhu, Xuening Sun, Hao Cheng, Song Dingjie, Zhihong Chen, Mosen Alharthi, Bang An, Juncai He, et al. 2024a. Acegpt, localizing large language models in arabic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 8139–8163.

Shijue Huang, Wanjun Zhong, Jianqiao Lu, Qi Zhu, Jiahui Gao, Weiwen Liu, Yutai Hou, Xingshan Zeng, Yasheng Wang, Lifeng Shang, et al. 2024b. Planning, creation, usage: Benchmarking llms for comprehensive tool utilization in real-world complex scenarios. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 4363–4400.

International Organization for Standardization. 2023a. *ISO/IEC 23894:2023: Information Technology, Artificial Intelligence, Guidance on Risk Management*. International Organization for Standardization.

International Organization for Standardization. 2023b. *ISO/IEC 42001:2023: Information Technology, Artificial Intelligence, Management System*. International Organization for Standardization.

Manish Joshi. 2026. Yalla at AISA-ArabicFC: Argument-focused fine-tuning and per-argument ensembling for Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Ali Khairallah and Arkaitz Zubiaga. 2026. QMUL_AK at AISA-ArabicFC: Schema-constrained function calling with ALLaM. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Toka Khaled Abdelfattah, Eman Elrefai, and Ahmed Soliman. 2026. Ṣaqr at AISA-ArabicFC: What actually influences Arabic function calling? In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Mayank Kulkarni, Vittorio Mazzia, Judith Gaspers, Chris Hench, Jack FitzGerald, and AGI Amazon. 2025. Massive-agents: A benchmark for multilingual function-calling in 52 languages. In *EMNLP (Findings)*, pages 20193–20215.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2024. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, volume 2024, pages 52989–53046.

Model Context Protocol. 2025. Model context protocol specification. https://modelcontextprotocol.io/specification.

Omer Nacar and Anis Koubaa. 2025. Enhancing semantic similarity understanding in arabic nlp with nested embedding learning. In *Generative AI and Large Language Models: Opportunities, Challenges, and Applications: Volume 1*, pages 179–216. Springer.

Omer Nacar, Anis Koubaa, Serry Sibaee, Yasser Al-Habashi, Adel Ammar, and Wadii Boulila. 2025. GATE: General Arabic text embedding for enhanced semantic textual similarity with matryoshka representation learning and hybrid loss training. *arXiv preprint arXiv:2505.24581*.

OpenAI. 2024. Openai structured outputs documentation.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2024. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations*, volume 2024, pages 9695–9717.

Neha Sengupta, Sunil Kumar Sahu, Bokang Jia, Satheesh Katipomu, Haonan Li, Fajri Koto, William Marshall, Gurpreet Gosal, Cynthia Liu, Zhiming Chen, et al. 2023. Jais and jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv preprint arXiv:2308.16149*.

Nader Hajed Shehri and Abdullah Aldosari. 2026. Wajeh at AISA-ArabicFC: A non-LLM classical ML and retrieval pipeline for Arabic function calling. In *Proceedings of the Fourth Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics. Co-located with EMNLP 2026, October 24–29.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, et al. 2025. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*.

Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. 2023. Autogen:

Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. Toolqa: A dataset for llm question answering with external tools. *Advances in Neural Information Processing Systems*, 36:50117–50143.

A Track B Leaderboard

Table 6 reports all 17 valid Track B team entries. The score is $0.30\,\textsc{FnAcc} + 0.50\,\textsc{ArgEM} + 0.20\,\textsc{ThinkRate}$. The leading three teams are unchanged from Track A, and 12 of 17 systems attain $\textsc{ThinkRate} = 1.0$. Four teams entered Track B without a Track A entry. For each team, the table reports the submitted run with the highest Track B score, subject to the limit of three runs per team per ranked track.

B Track C: Cross-Dialect Robustness

Table 7 orders the dialect diagnostic by equal-dialect macro score. QMUL_AK obtains the highest value, 0.9075, followed by Al-Shahiq at 0.9031 and Ṣaqr at 0.8994. Restricting the comparison to teams also entering Track A, the reordering occurs among the leading six; the relative order of the remaining shared entrants is unchanged. Track-B-only teams change their absolute positions in the full table.

The diagnostic uses final test dialect labels: 660 MSA, 175 Egyptian, 128 Gulf, 128 Levantine, and 34 Maghrebi rows. The released input file contains earlier labels, including only 11 Maghrebi rows, and should not be used to recover the final scoring denominators. Appendix D records the 179 reassignments.

Maghrebi is the lowest-scoring group for all 21 systems, and its best score, 0.859, is below the median score of each other group (Figure 3). This pattern motivates further evaluation, but does not isolate a causal effect of dialect: dialect subsets can differ in tools, argument patterns, no-call prevalence, and annotation properties. The small Maghrebi subset also makes close macro-score comparisons sensitive to few outcomes. Specifically, one additional argument match in a dialect with $N_{+,g}$ positive rows changes the macro score by $0.12/N_{+,g}$ when function correctness is unchanged. We report the diagnostic ordering descriptively and make no statistical-significance claim about close differences.

C Additional Discussion Evidence

Intervention inventory. Table 8 summarizes interventions credited with a gain in the 13 system descriptions analyzed in this paper. A method is counted only when the description reports its use in the submitted system and attributes an improvement to it; methods tested and rejected are excluded. A team can contribute to several categories. These counts summarize participant reports, not organizer-run ablations, and the reported effect sizes below retain their original evaluation settings.

Training signal and output shaping. Yalla (Joshi, 2026) reports that the scored object constitutes only 30.3% of the distributed target and obtains a 0.020 absolute increase in ArgEM by supervising that object alone, exceeding the gain from its subsequent reinforcement-learning stage. Saudi Legal FC (Almohammedi, 2026) improves development ArgEM from 0.732 to 0.778 using completion-only loss masking. Thakaa (Alqaeri et al., 2026) reports a 7.5-percentage-point gain from explicitly stating annotation conventions in the prompt. Ṣaqr (Khaled Abdelfattah et al., 2026) reports inference-time gains from reasoning prefill and schema-typed parsing without retraining. These within-team comparisons motivate attention to supervision and output representation, but their magnitudes are not directly comparable across different models and evaluation settings.

Failure localization. Arabic Function Forge (Alshehri and Aljubayri, 2026) identifies over-interpretation as a recurring error: translating argument values, changing date representations, or supplying fields absent from the request. PuDyDG (Dang Phu and Dang Van, 2026) reports that 55 of 78 development argument failures (70.5%) retain the correct key set and differ in one or more values. ArgTune (Baothman, 2026) reports incorrect argument dictionaries on 93 of the 500 positive development examples, with the correct tool predicted in all 93 cases. These analyses locate substantial remaining error in argument values under the annotation and normalization policy; they do not independently establish whether every alternative prediction would fail in a downstream service.

Diminishing returns from additional complexity. Ṣaqr reports improvements from 270M through 4B under its tested configurations, without a corresponding improvement from every larger model.

Rank	Team	Overall	FnAcc	ArgEM	ThinkRate
1	Ṣaqr	0.9400	0.9973	0.8816	1.0000
2	Yalla	0.9395	0.9973	0.8806	1.0000
3	QMUL_AK	0.9380	0.9938	0.8796	1.0000
4	Arabic Function Forge	0.9311	0.9893	0.8686	1.0000
5	Thakaa	0.9306	0.9893	0.8676	1.0000
6	PuDyDG	0.9260	0.9973	0.8536	1.0000
7	Sa74ll	0.9149	0.9947	0.8455	0.9689
8	Durham-Jazan	0.9109	0.9938	0.8255	1.0000
9	Makeen	0.9106	0.9911	0.8265	1.0000
10	Siraj	0.9101	0.9929	0.8245	1.0000
11	Wukala Arabiyya	0.9066	0.9778	0.8265	1.0000
12	Saudi Legal FC	0.8946	0.9831	0.7994	1.0000
13	farah	0.8936	0.8862	0.8576	0.9947
14	GymMinds	0.8718	0.9964	0.7713	0.9360
15	Wajeh	0.8282	0.9138	0.7081	1.0000
16	TIGER	0.8037	0.9929	0.6690	0.8569
17	Optimal	0.7137	0.9324	0.5838	0.7102

Table 6: Complete Track B leaderboard using the final gold and official scorer. Overall is $0.30\,\mathrm{FnAcc}+0.50\,\mathrm{ArgEM}+0.20\,\mathrm{ThinkRate}$. Trace compliance measures the presence of a qualifying string, not reasoning quality.

Thakaa evaluates rules, retrieval, prompting, selection, reranking, and fusion without obtaining a reliably replicated gain beyond its fine-tuning reference. For 60.7% of its remaining development failures, the gold argument dictionary appears in a pool of 20 samples, but the tested selectors do not reliably recover it. Siraj (Alanazi and Alshammari, 2026) rejects five candidate post-editing rules evaluated on out-of-fold predictions; the retained post-editing stage yields only a small reported blind-test improvement. These findings concern the tested recipes and data, rather than establishing a general disadvantage of larger models or more complex pipelines.

Reasoning traces. Twelve of the 17 Track B systems attain $\textsc{ThinkRate} = 1.0$. Yalla reports that, for 97.7% of rows, its trace is attached after prediction from a separate reasoning-trained model whose tool call matches the submitted call. Other systems use deterministic or separately constructed traces. The official length-based check does not verify Arabic language, trace–call consistency, faithfulness, or the trace's role in producing the prediction. High compliance therefore supports a claim about output format, not the causal benefit of reasoning.

Dialect handling. QMUL_AK (Khairallah and Zubiaga, 2026) incorporates minority-dialect resampling and obtains the highest Track C macro score, with a dialect gap of 0.077. The contribution of resampling is not isolated by an ablation. Saudi Legal FC instead attributes many inspected Levantine errors to enumeration and date-format patterns also found in MSA. These observations motivate distinguishing linguistic coverage from differences in argument patterns and annotation conventions across dialect subsets.

D Test Annotation Audit

Review scope. The archived review file marks all $1{,}125$ test instances as reviewed and contains 173 distinct written rationale strings. The review interface displays the request, candidate schemas, reference function, and arguments, and saves edits to a working copy. Comments document both dialect-label normalization and dialect reassignment. The distinct strings are review notes; they do not represent independent annotations or a reviewer count. The retained records do not identify reviewers, establish their expertise, or document independently duplicated judgments. Consequently, no inter-annotator agreement statistic is reported, and the recorded test review is not extended into a claim of complete human verification of the training and development data.

Gold-version changes. Comparing the v1.6 test annotations with the final scoring gold identifies 419 changed argument dictionaries and eight changed function labels (Table 9). These counts refer to that specific test-version comparison. A row may have both changes, and the counts are not a breakdown of all original schema faults in the source collection.

Dialect relabeling. The final Track C labels differ from the released test labels on 179 rows. Table 10 gives every transition, and Table 11 gives the re-

Rank	Team	Macro	Δ	MSA	Gulf	Egyptian	Levantine	Maghrebi	Gap
1	QMUL_AK	0.9075	+2	0.935	0.915	0.932	0.897	0.859	0.077
2	Al-Shahiq	0.9031	+2	0.924	0.934	0.925	0.891	0.841	0.093
3	Ṣaqr	0.8994	−2	0.942	0.913	0.935	0.902	0.806	0.136
4	Thakaa	0.8978	+2	0.929	0.910	0.903	0.901	0.847	0.082
5	Yalla	0.8957	−3	0.946	0.892	0.928	0.907	0.806	0.140
6	Arabic Function Forge	0.8847	−1	0.933	0.890	0.933	0.880	0.788	0.145
7	PuDyDG	0.8800	0	0.930	0.902	0.904	0.876	0.788	0.142
8	ArgTune	0.8754	0	0.920	0.918	0.918	0.886	0.735	0.184
9	Sa74ll	0.8751	*n/a*	0.922	0.923	0.891	0.862	0.776	0.147
10	Siraj	0.8703	*n/a*	0.906	0.922	0.852	0.883	0.788	0.134
11	Durham-Jazan	0.8678	0	0.911	0.901	0.856	0.883	0.788	0.123
12	Wukala Arabiyya	0.8658	0	0.894	0.919	0.859	0.904	0.753	0.166
13	Makeen	0.8647	*n/a*	0.905	0.870	0.901	0.878	0.771	0.134
14	MOSKA-LLM	0.8592	0	0.888	0.877	0.869	0.874	0.788	0.100
15	Saudi Legal FC	0.8547	0	0.884	0.913	0.836	0.864	0.776	0.136
16	farah	0.8464	0	0.868	0.872	0.916	0.859	0.718	0.198
17	GymMinds	0.8335	0	0.876	0.865	0.828	0.881	0.718	0.163
18	ToolCallers	0.8247	0	0.868	0.841	0.821	0.858	0.735	0.132
19	TIGER	0.7804	*n/a*	0.810	0.807	0.779	0.788	0.718	0.092
20	Wajeh	0.7661	0	0.797	0.798	0.798	0.791	0.647	0.151
21	Optimal	0.6950	0	0.739	0.685	0.726	0.713	0.612	0.128
	Mean	*0.8546*		*0.8918*	*0.8794*	*0.8719*	*0.8656*	*0.7647*	*0.1335*

Table 7: Track C diagnostic, ordered by the unweighted mean of five within-dialect Track A scores. Δ compares ranks after restricting both orderings to shared Track A entrants; *n/a* denotes no Track A entry. Gap is the highest minus lowest dialect score and does not determine the ordering. Final test dialect labels are used.

Intervention credited with a gain	Teams
Deterministic post-processing or repair	12
Data augmentation or resampling	6
Prompt or schema formatting	5
Training-objective or loss shaping	4
Reasoning-trace handling	3
Ensembling or voting	2
Dialect-specific handling	1

Table 8: Interventions credited with a gain in the 13 analyzed system descriptions. Categories overlap.

Comparison	Rows
Argument dictionaries: v1.6 test gold to final gold	419
Function labels: v1.6 test gold to final gold	8
Dialect labels: released test inputs to final gold	179

Table 9: Version-specific test annotation changes. Counts can overlap and must not be summed as unique affected instances.

sulting counts. Relabeling does not change test membership. The final Maghrebi subset contains 34 rows; the 11 rows in the released input labels are not its official scoring denominator. Within each dialect, FnAcc uses all rows, whereas ArgEM uses only positive rows.

Call-label composition. The final test gold contains 997 call rows and 128 no-call rows. Training and development contain 50 and 45 no-call rows, respectively. Their no-call proportions are 0.47%, 8.26%, and 11.38% across train, development, and test, so abstention prevalence differs substantially across splits. The corpus contains 223 no-call rows under the final test labels and eleven recorded abstention categories. The 11,095 released reasoning traces belong to training and development.

Released	Final	Rows
MSA	Egyptian	75
MSA	Gulf	53
MSA	Levantine	27
MSA	Maghrebi	10
Gulf	Maghrebi	9
Egyptian	Maghrebi	4
Levantine	Gulf	1
Total		**179**

Table 10: All test dialect-label transitions between the released input labels and the final scoring labels.

E Recoverable Baseline Configuration

GPT-4o settings. Table 12 records the settings preserved in the baseline inference script. The three-shot condition uses three manually authored demonstrations, including one no-call example; they were not sampled from the training split. No claim is made that these demonstrations balance dialects or domains.

Prompt and generation reproducibility. The original inference script contains the system prompt

Dialect	Released	Final
MSA	825	660
Egyptian	104	175
Gulf	83	128
Levantine	102	128
Maghrebi	11	34
Total	**1,125**	**1,125**

Table 11: Test row counts before and after dialect relabeling. Final counts include positive and no-call rows; they are not necessarily the positive-only denominators for ArgEM.

Setting	Recorded value
Model alias	`gpt-4o`
Temperature	0
Output limit	`max_tokens=300`
Demonstrations	Zero, or three manually authored examples
Negative example	One in the three-shot condition
Retry setting	`max_retries=3`, exponential backoff
Parsing	Markdown-fence removal and regular-expression JSON fallback
Failure default	Abstention (`none`)

Table 12: Recoverable GPT-4o baseline settings. The alias is not pinned to an identifiable historical model snapshot.

and the three demonstrations, but an unpinned `gpt-4o` alias does not identify the exact historical API snapshot. Consequently, matching the prompt and decoding settings does not guarantee regeneration of the archived predictions. The three-shot result characterizes the recorded demonstration set; it is not evidence that few-shot prompting generally degrades performance.

The 270M baseline. The released AISA-AR-FunctionCall-Think model card identifies `google/gemma-3-270m` as the base model and describes LoRA adaptation with rank 64, three training epochs, and a cosine learning-rate scheduler.[4] These are model-card statements; the original complete training configuration was not recovered. They do not establish the missing optimizer, learning-rate, batch-size, seed, and checkpoint-selection details required for a complete retraining specification.

Archived random predictions. The random-baseline prediction file remains available for rescoring, but its generator was not recovered. Its reported FnAcc of 0.0418 should therefore not be described as the chance performance of uniform selection from four candidates. For a rule that always calls and samples uniformly from the four candidates,

$$\mathbb{E}[\text{FnAcc}] = \frac{997}{1125} \cdot \frac{1}{4} \approx 0.2216. \tag{8}$$

This expectation assumes one gold candidate on each positive row and no correct candidate on a no-call row. The historical random result is retained as an archived run, not a fully specified sampling algorithm.

Score reproduction. Rescoring the four archived baseline prediction files with the final gold and scorer reproduces their reported metrics. Rescoring the archived QMUL_AK predictions similarly reproduces FnAcc 0.9938, ArgEM 0.8796, Track A 0.9253, Track B 0.9380, and Track C 0.9075. This verifies the scoring of those predictions; it does not constitute fresh inference from the original models. Independent blind-test rescoring requires the exact final gold, prediction files, scorer, and normalizer revisions.

F Additional Evaluation Diagnostics

Participation and selected runs. Twenty-two teams submitted to the blind phase. One team's Track A and Track B files contained no parseable predictions and were recorded as invalid with zero scores, leaving 21 valid teams. The 17 valid entries in each ranked track share 13 teams; four teams appear only in Track A and four only in Track B. The submission record contains 73 entries. Each track selects the highest-scoring eligible run for each team using its own Overall score.

Analytical selection shortcuts. The first candidate is the gold tool on all 997 positive test rows, so always choosing that position yields FnAcc $997/1125 = 0.8862$. In the released inputs, `negative_category` is non-null on 125 rows, all negative under the final gold. Its precision as a no-call indicator is 100% and its recall is $125/128 = 97.66\%$. Predicting `none` when the field is non-null, and otherwise selecting the first candidate, therefore gives

$$\text{FnAcc}_{\text{shortcut}} = \frac{125 + 997}{1125} = 0.9973. \tag{9}$$

This analytical count matches the best reported participant FnAcc without interpreting the request. It is a diagnostic of the released inputs, not evidence that a particular participant used the rule. Neither shortcut supplies argument predictions.

[4] https://huggingface.co/TuwaiqAcademy/AISA-AR-FunctionCall-Think

Abstention error decomposition. For QMUL_AK, the seven function-label errors comprise six false calls on 128 negative rows and one missed call on 997 positive rows. Their rates are 4.69% and 0.10%, respectively. Among the 996 positive rows for which the system predicts a call, there are no wrong-tool errors. This decomposition concerns one archived system output, and the input shortcuts limit semantic conclusions from its high function-label accuracy.

Leading-system margins. Ṣaqr, Yalla, and QMUL_AK match 879, 878, and 877 argument dictionaries, respectively, out of 997 positive rows. Ṣaqr and Yalla have identical function-label accuracy; their net difference of one argument match contributes $0.60/997 = 0.000602$ to Track A and $0.50/997 = 0.000502$ to Track B, where both also have full trace compliance. A one-count difference does not mean that the systems disagree on only one instance. Without a paired analysis of their predictions, it also does not establish statistical equivalence or a significant performance difference.

Dialect-score sensitivity. In Track C, changing one argument outcome in dialect g changes the macro score by $0.12/N_{+,g}$ when function correctness is unchanged; changing one function-label outcome changes it by $0.08/N_g$ when argument correctness is unchanged. Here N_g is the total dialect count and $N_{+,g}$ its positive count. The two contributions add when both outcomes change. Small dialect subsets therefore give individual examples more influence. Total row counts alone are insufficient to compute argument-level sensitivity without the positive denominators. The reported diagnostic ordering is descriptive; no paired confidence intervals or significance tests are claimed.